



ELSEVIER

Available online at www.sciencedirect.com

SCIENCE @ DIRECT®

Electronic Notes in Discrete Mathematics 19 (2005) 133–139

Electronic Notes in
DISCRETE
MATHEMATICS

www.elsevier.com/locate/endm

An all-substrings common subsequence algorithm¹

C. E. R. Alves²

*Faculdade de Tecnologia e Ciências Exatas
Universidade São Judas Tadeu
São Paulo, Brazil*

E. N. Cáceres³

*Departamento de Computação e Estatística
Universidade Federal de Mato Grosso do Sul
Campo Grande, Brazil*

S. W. Song⁴

*Departamento de Ciência da Computação
Universidade de São Paulo
São Paulo, Brazil*

Abstract

Given two strings A and B of lengths n_a and n_b , respectively, the *all-substrings longest common subsequence (ALCS)* problem obtains, for every substring B' of B , the length of the longest string that is a subsequence of both A and B' . We present an algorithm for this problem that takes $O(n_a n_b)$ time and $O(n_b)$ space. To our knowledge, this is the first algorithm in the literature for the ALCS problem.

Keywords: all substrings common subsequence, string processing

1 Introduction

The computation of patterns in strings is a fundamental problem in Molecular Biology and many other areas [8]. In this paper we consider a variant of the longest common subsequence problem. Consider a string of symbols from a finite alphabet. A *substring* of a string is any fragment of contiguous symbols of the string. A *subsequence* of a string is obtained by deleting zero or more symbols from the original string, at any position. Given strings A and B , $|A| = n_a$ and $|B| = n_b$, the *longest common subsequence (LCS)* problem obtains the longest subsequence common to both. A generalization of the LCS problem is the *all-substrings longest common subsequence (ALCS)* problem that obtains the lengths of all the longest common subsequences for string A and *all substrings* of B . The ALCS problem is a restricted form of the all-substrings alignment problem [1,7]. It has many applications, such as finding approximate tandem repeats in strings [7], solving the circular alignment of two strings [6,7] and finding the alignment of one string with several others that have a common substring [5]. Sequential algorithms for the LCS problem have been proposed in [2,4]. LCS can be solved in $O(n_a n_b)$ time. There is, however, no sequential algorithm for the ALCS problem in the literature. Schmidt [7] presents an algorithm to solve the string edit problem to find the optimal weighted alignments between substrings of A and substrings of B . Based on [7], we present an algorithm to solve the ALCS problem that takes $O(n_a n_b)$ time and $O(n_b)$ space.

2 The all-substrings longest common subsequence algorithm

Consider a string of symbols from a finite alphabet. A *substring* of a string is any fragment of contiguous symbols of the string. A *subsequence* of a string is obtained by deleting zero or more symbols from the original string, at any position. The remaining symbols preserve the order they had on the original string. Strings of symbols will be denoted by upper-case letters. Individual symbols of a string will be identified by the string name (lower-case) with a

¹ Partially supported by FINEP-PRONEX-SAI Proc. 76.97.1022.00, CNPq Proc. 30.0317/02-6, 30.5218/03-4, 47.0163/03-8, 55.2028/02-9, and FUNDECT-MS Proc. 41/100117/03. We also thank the anonymous referees for their helpful comments.

² Email: prof.carlos_r.alves@usjt.br

³ Email: edson@dict.ufms.br

⁴ Email: song@ime.usp.br

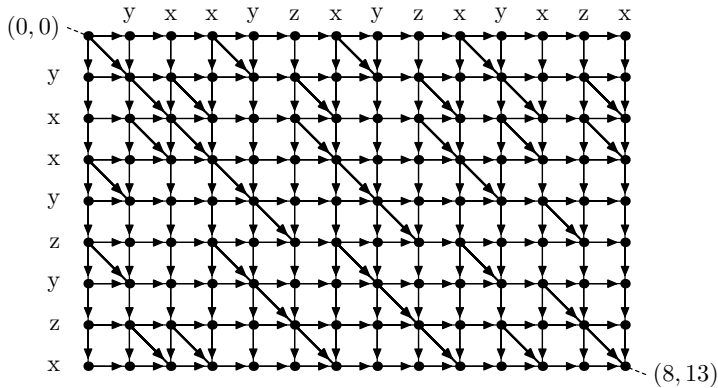


Fig. 1. GDAG for the ALCS problem.

subscripted index. The indices start at 1, for the first symbol. The length of the string will be denoted by n subscripted by the lower-case string name. For example, $A = a_1a_2a_3 \dots a_{n_a-1}a_{n_a}$. A substring is denoted by the upper-case letter of the original string and indices that indicate the initial (subscripted) position and final (superscripted) position of the symbols in the original string. For example, $A_3^6 = a_3a_4a_5a_6$. Given two strings A and B , the longest common subsequence (LCS) problem finds the longest string C that is a subsequence of A and B .

For two strings A and B , $|A| = n_a$ and $|B| = n_b$, the *all-substrings longest common subsequence* (ALCS) problem solves the LCS problem for A and all substrings of B , that is, solve the LCS for all pairs (A, B_i^j) , where $1 \leq i \leq j \leq n_b$. Often we are more interested in the *lengths* rather than the subsequences proper.

We associate a *Grid Directed Acyclic Graph* (GDAG) to the ALCS problem. It has $n_a + 1$ rows and $n_b + 1$ columns of vertices. Label the rows from 0 to n_a and the columns from 0 to n_b . Each vertex of a GDAG G is identified by $G(i, j)$, where $0 \leq i \leq n_a$ and $0 \leq j \leq n_b$. The arcs in this GDAG have weights 0 or 1: i) (diagonal arcs) for $1 \leq i \leq n_a$ and $1 \leq j \leq n_b$ there is an arc from $G(i-1, j-1)$ to $G(i, j)$ with weight 1 if $a_i = b_j$, 0 otherwise; ii) (vertical arcs) for $0 \leq i \leq n_a$ and $1 \leq j \leq n_b$ there is an arc from $G(i, j-1)$ to $G(i, j)$ with weight 0; iii) (horizontal arcs) for $1 \leq i \leq n_a$ and $0 \leq j \leq n_b$ there is an arc from $G(i-1, j)$ to $G(i, j)$ with weight 0. Fig. 1 shows an example.

Denote the vertices at the top row of G by $T_G(i)$ and the ones at the bottom row by $F_G(i)$. To solve the LCS problem, one looks for the *highest weighted path* from $G(0, 0)$ to $G(n_a, n_b)$, whose total weight is the length of the longest common subsequence between A and B . This path gives us the actual

subsequence: each vertical arc represents a deletion in A , each horizontal arc represents a deletion in B and each diagonal arc (of weight 1) represents matched symbols in A and B . A Dynamic Programming technique is used in the LCS algorithm by Hirschberg [3] of $O(n_a n_b)$ time and $O(n_a + n_b)$ space.

For the ALCS problem we are interested in more paths than just the one from $G(0, 0)$ to $G(n_a, n_b)$. We define $C_G(i, j)$ as follows. For $0 \leq i \leq j \leq n_b$, $C_G(i, j)$ is the largest total weight of a path from $T_G(i)$ to $F_G(j)$ in G . If $i > j$, there is no path between $T_G(i)$ and $F_G(j)$, so we make $C_G(i, j) = 0$. $C_G(i, j)$ represents the length of the longest common subsequence of A and the substring B_{i+1}^j . When $i = j$, there is no such substring and the only path from $T_G(i)$ to $F_G(j)$ is formed exclusively by vertical arcs and has total weight 0. When $i > j$, B_{i+1}^j also does not exist and we adopt $C_G(i, j) = 0$. The values of $C_G(i, j)$ have the following interesting properties. The proofs can be found in the complete paper.

Properties 2.1

- (i) For all i ($0 \leq i \leq n_b$) and all j ($1 \leq j \leq n_b$) we have $C_G(i, j) = C_G(i, j-1)$ or $C_G(i, j) = C_G(i, j-1) + 1$.
- (ii) For all i ($1 \leq i \leq n_b$) and all j ($0 \leq j \leq n_b$) we have $C_G(i-1, j) = C_G(i, j)$ or $C_G(i-1, j) = C_G(i, j) + 1$.
- (iii) For all i and all j ($1 \leq i < j \leq n_b$) if $C_G(i-1, j) = C_G(i-1, j-1) + 1$ then $C_G(i, j) = C_G(i, j-1) + 1$.

Consider the GDAG G for the ALCS problem. We define D_G as follows. For $1 \leq k \leq n_a$, $D_G(i, k)$ indicates the value of j such that $C_G(i, j) = k$ and $C_G(i, j-1) = k-1$. If there is no such value, $D_G(i, k) = \infty$. We define $D_G(i, 0) = i$. We denote D_G^i the row i of D_G . As example, we show below $C_G(i, j)$ and $D_G(i, k)$ corresponding to the GDAG of Fig. 1. From the values of $D_G(i, k)$ we can obtain the values of $C_G(i, j)$.

Properties 2.2 For $0 \leq i < n_b$,

- (i) There is one unique finite component of D_G^i that does not appear in D_G^{i+1} , which is $D_G(i, 0) = i$.
- (ii) There is at most one finite component of D_G^{i+1} that does not appear in D_G^i .

j	0	1	2	3	4	5	6	7	8	9	10	11	12	13
$C_G(0, j)$	0	1	2	3	4	5	6	6	7	8	8	8	8	8
$C_G(1, j)$	0	0	1	2	3	4	5	5	6	7	7	7	7	7
$C_G(2, j)$	0	0	0	1	2	3	4	4	5	6	6	6	6	7
$C_G(3, j)$	0	0	0	0	1	2	3	3	4	5	5	6	6	7
$C_G(4, j)$	0	0	0	0	0	1	2	2	3	4	4	5	5	6
$C_G(5, j)$	0	0	0	0	0	0	1	2	3	4	4	5	5	6
$C_G(6, j)$	0	0	0	0	0	0	0	1	2	3	3	4	4	5
$C_G(7, j)$	0	0	0	0	0	0	0	0	1	2	2	3	3	4
$C_G(8, j)$	0	0	0	0	0	0	0	0	0	1	2	3	3	4
$C_G(9, j)$	0	0	0	0	0	0	0	0	0	0	1	2	3	4
$C_G(10, j)$	0	0	0	0	0	0	0	0	0	0	0	1	2	3
$C_G(11, j)$	0	0	0	0	0	0	0	0	0	0	0	0	1	2
$C_G(12, j)$	0	0	0	0	0	0	0	0	0	0	0	0	0	1
$C_G(13, j)$	0	0	0	0	0	0	0	0	0	0	0	0	0	0

k	0	1	2	3	4	5	6	7	8	j	$V_G(j)$
$D_G(0, k)$	0	1	2	3	4	5	6	8	9		
$D_G(1, k)$	1	2	3	4	5	6	8	9	∞	1	∞
$D_G(2, k)$	2	3	4	5	6	8	9	13	∞	2	13
$D_G(3, k)$	3	4	5	6	8	9	11	13	∞	3	11
$D_G(4, k)$	4	5	6	8	9	11	13	∞	∞	4	∞
$D_G(5, k)$	5	6	7	8	9	11	13	∞	∞	5	7
$D_G(6, k)$	6	7	8	9	11	13	∞	∞	∞	6	∞
$D_G(7, k)$	7	8	9	11	13	∞	∞	∞	∞	7	∞
$D_G(8, k)$	8	9	10	11	13	∞	∞	∞	∞	8	10
$D_G(9, k)$	9	10	11	12	13	∞	∞	∞	∞	9	12
$D_G(10, k)$	10	11	12	13	∞	∞	∞	∞	∞	10	∞
$D_G(11, k)$	11	12	13	∞	∞	∞	∞	∞	∞	11	∞
$D_G(12, k)$	12	13	∞	∞	∞	∞	∞	∞	∞	12	∞
$D_G(13, k)$	13	∞	∞	∞	∞	∞	∞	∞	∞	13	∞

Properties 2.2(i) and 2.2(ii) indicate that we can transform one row of D_G into the next row by the removal of one component (the first one) and the insertion of another component (finite or not). This suggests a representation for D_G in $O(n_a + n_b)$ space. This representation is composed by D_G^0 (the first row of D_G), that has size $n_a + 1$, and a vector V_G of size n_b , such that, for $1 \leq i \leq n_b$, $V_G(i)$ is the value of the finite component that is present in row D_G^i but not in row D_G^{i-1} . If there is no such component, then $V_G(i) = \infty$. D_G^0 (first row of D_G) and V_G can be used together to reconstruct D_G .

In our complete paper we present additional properties. By such properties, we can encode *variations* in the weights of the paths to internal vertices of the GDAG in an efficient way. If we pick a vertex on the upper row of G , say $T_G(i)$, and determine the best possible paths from it to two neighbor vertices on a same row, say $(l, j - 1)$ and (l, j) , these paths will have the same weight if i is below a certain limit. For values of i equal or above this limit, the weight of the path to (l, j) will be larger (by 1) than the weight of the path to

Algorithm 1 The ALCS Algorithm

```

1: for  $j \leftarrow 0$  to  $n_b$  do
2:    $i_h(0, j) \leftarrow j$ 
3: end for
4: for  $l \leftarrow 0$  to  $n_a$  do
5:    $i_v(l, 0) \leftarrow 0$ 
6: end for
7: for  $l \leftarrow 1$  to  $n_a$  do
8:   for  $j \leftarrow 1$  to  $n_b$  do
9:     if  $a_l \neq b_j$  then
10:       $i_h(l, j) \leftarrow \max(i_v(l, j-1), i_h(l-1, j))$ 
11:       $i_v(l, j) \leftarrow \min(i_v(l, j-1), i_h(l-1, j))$ 
12:     else
13:       $i_h(l, j) \leftarrow i_v(l, j-1)$ 
14:       $i_v(l, j) \leftarrow i_h(l-1, j)$ 
15:     end if
16:   end for
17: end for
18: for  $j \leftarrow 1$  to  $n_b$  do
19:    $V_G(j) \leftarrow \infty$ 
20: end for
21:  $D_G^0(0) \leftarrow 0$ 
22:  $i \leftarrow 1$ 
23: for  $j \leftarrow 1$  to  $n_b$  do
24:   if  $i_h(n_a, j) = 0$  then
25:      $D_G^0(i) \leftarrow j$ 
26:      $i \leftarrow i + 1$ 
27:   else
28:      $V_G(i_h(n_a, j)) \leftarrow j$ 
29:   end if
30: end for
31: for  $l \leftarrow i$  to  $n_a$  do
32:    $D_G^0(l) \leftarrow \infty$ 
33: end for

```

$(l, j-1)$. The value of this limit depends on l and j and will be called $i_h(l, j)$. Similarly, the weights of the best paths from $T_G(i)$ to neighbor vertices on a column, $(l-1, j)$ and (l, j) , differ by 1 if i is *below* a certain limit $i_v(l, j)$. If i is on or above this limit, the paths have the same weight.

The determination of i_h and i_v is done vertex by vertex, by sweeping the GDAG by rows (or columns). To calculate the limits for a vertex (l, j) it is necessary to know only $i_h(l-1, j)$ and $i_v(l, j-1)$, for these values indicate differences between paths to vertices adjacent to (l, j) . The proposed algorithm is shown in Algorithm 1. Theorem 2.1 summarizes the main result.

Theorem 2.1 *Given two strings A and B , of lengths n_a and n_b respectively, it is possible to solve the ALCS problem for A and B in time $O(n_a n_b)$ and space $O(n_b)$ (or $O(n_a n_b)$ to allow the retrieval of the actual best paths).*

References

- [1] Alves, C.E.R., E. N. Cáceres, F. Dehne, and S. W. Song. *Parallel dynamic programming for solving the string editing problem on a CGM/BSP*. Proceedings of the 14th ACM Symposium on Parallel Algorithms and Architectures - SPAA 2002, ACM Press (2002) 275–281.
- [2] Apostolico, A., and C. Guerra. *The longest common subsequence problem revisited*. Algorithmica **2** (1987) 315–336.
- [3] Hirschberg, D.S. *A linear space algorithm for computing maximal common subsequences*. Communications of the ACM **18** (1975) 341–343.
- [4] Hunt, J.W., and T. G. Szymansky. *A fast algorithm for computing longest common subsequences*. Communications of the ACM **20** (1977) 350–353.
- [5] Landau, G.M., and M. Ziv-Ukelson. *On the common substring alignment problem*. Journal of Algorithms **41** (2001) 338–359.
- [6] Maes, M. *On a cyclic string-to-string correction problem*. Information Processing Letters **35** (1990) 73–78.
- [7] Schmidt, J. *All highest scoring paths in weighted graphs and their application to finding all approximate repeats in strings*. SIAM J. Computing **27** (1998) 972–992.
- [8] Smyth, W. “Computing Patterns in Strings”. Pearson - Addison Wesley, 2003.